Rapport de travaux pratiques
OpenLDAP 2.6 sous Docker (Debian 12)

Thomas Broine

TP LDAP - Dépôt `TP-LDAP`

Résumé

Ce document guide pas à pas la réalisation du TP LDAP : déploiement conteneurisé d'OpenLDAP 2.6, conception du DIT, rôles et ACL, intégration Linux (NSS/PAM), fédération avec Keycloak, réplication lecture/écriture vers un réplica en lecture seule, puis méta-annuaire agrégeant deux annuaires. Les commandes et vérifications sont reprises pour être exécutées telles quelles sur une machine hôte équipée de Docker et, pour les tests LDAP, du paquet `ldap-utils` (ou équivalent).

Table des matières

1 Introduction et objectifs **3**
1.1 Cadre du TP . . . 3
1.2 Prérequis sur la machine hôte . . . 3
1.3 Convention de travail dans le dépôt . . . 3
1.4 Rappel sur OpenLDAP et le DIT . . . 4

2 Installation et déploiement automatisé **5**
2.1 Recherche de la cible à déployer . . . 5
2.2 Construction et démarrage . . . 5
2.3 Comportement au démarrage du conteneur . . . 5
2.4 Persistance . . . 5
2.5 Vérifications (suffixe et service) . . . 6
2.6 Recherche d'informations sur la configuration (cn=config) . . . 6
2.7 Tests automatisés - `test_01_installation_deploiement.sh` . . . 6

3 Conception de la structure DIT **7**
3.1 Contexte . . . 7
3.2 Paramètres du base DN . . . 7
3.3 Arborescence retenue . . . 7
3.4 Exemple de LDIF conceptuel (racine et OU) . . . 7
3.5 Vérifications . . . 8
3.6 Justification (éléments de rapport) . . . 8
3.7 Tests automatisés - `test_02_conception_dit.sh` . . . 8

4 Discrétisation des rôles et ACL **9**
4.1 Groupes et rôles . . . 9
4.2 Compte `cn=admin` et administration courante . . . 9
4.3 Où sont les ACL ? . . . 9
4.4 Règles déployées dans le dépôt . . . 9
4.5 Validation manuelle recommandée . . . 10
4.6 Tests automatisés - `test_03_roles_acl.sh` . . . 10

5 Intégration Linux (NSS / PAM) **11**
5.1 Finalité . . . 11
5.2 Prérequis annuaire : attributs POSIX . . . 11
5.3 Configuration dans le conteneur . . . 11
5.4 Vérifications dans le conteneur . . . 11
5.5 Authentification (`su`, SSH) . . . 12
5.6 Tests automatisés - `test_04_integration_linux.sh` . . . 12

6 Intégration Keycloak (OpenID) **13**
6.1 Déploiement Keycloak 13
6.2 User Federation LDAP 13
6.3 Vérifications 13
6.4 Tests automatisés - `test_05_keycloak.sh` 14

7 Réplication LDAP (RW / RO) **15**
7.1 Architecture 15
7.2 Séquence d'initialisation (résumé) 15
7.3 Vérifications manuelles 15
7.4 Tests automatisés - `test_06_replication.sh` 16
7.5 Dépannage (rappel) 16

8 Fédération LDAP (méta-annuaire) **17**
8.1 Principe 17
8.2 Services dans le dépôt 17
8.3 Vue côté client 17
8.4 Tests automatisés - `test_07_meta_annuaire.sh` 17

9 Bilan : suite de tests complète **19**

Chapitre 1

Introduction et objectifs

1.1 Cadre du TP

Le cahier des charges est défini dans `INSTRUCTIONS.md` à la racine du dépôt. Les objectifs pédagogiques sont les suivants :

1. Comprendre l'architecture et le fonctionnement d'un annuaire LDAP.
2. Déployer un serveur OpenLDAP 2.6 dans un environnement conteneurisé (image Debian 12, paquets `slapd` et `ldap-utils`).
3. Concevoir une structure d'annuaire (DIT) adaptée au cas d'usage.
4. Mettre en œuvre une gestion fine des droits (discrétisation des rôles, ACL).
5. Intégrer LDAP avec Linux (NSS/PAM).
6. Fédérer LDAP avec Keycloak (OpenID).
7. Mettre en place une réplication (fournisseur RW, consommateur RO).
8. Créer un méta-annuaire agrégeant plusieurs sources LDAP.
9. Documenter et tester l'infrastructure (scripts fournis dans `projet/test/`).

1.2 Prérequis sur la machine hôte

— **Docker** et **Docker Compose** (plugin `docker compose`).

— **Ports libres** : `389` (LDAP principal), `1389` (réplica), `2389` (annuaire Acme), `3389` (méta-annuaire), `8090` et `9001` (Keycloak). Si `8090` est pris, adapter `docker-compose.yml` ou la variable `KEYCLOAK_URL` pour les tests.

— **Clients LDAP** sur l'hôte : paquet `ldap-utils` (Debian/Ubuntu) ou `openldap-clients` (Fedora/RHEL) pour `ldapsearch`, `ldapwhoami`, etc.

— Pour Keycloak : `curl` et `python3` (scripts de configuration et tests).

1.3 Convention de travail dans le dépôt

Toutes les commandes `docker compose` s'exécutent depuis le répertoire `projet/` du dépôt (contexte de build et fichier `docker-compose.yml`). Depuis la racine du dépôt, on peut utiliser `docker compose -f projet/docker-compose.yml`.

NOTA : Dans ce rapport, le suffixe par défaut est `dc=example,dc=org`, le mot de passe administrateur annuaire `admin` (`LDAP_ADMIN_PASSWORD`), et les utilisateurs de démo `thomas` / `thomas123` et `john` / `john123`, conformément au `docker-compose.yml` fourni.

1.4 Rappel sur OpenLDAP et le DIT

On distingue :

— `cn=config` : configuration dynamique du serveur (bases MDB, modules, ACL `olcAccess`, overlays). L'administration locale se fait souvent avec `ldapmodify -Y EXTERNAL -H ldapi:///` depuis le conteneur.

— **Le suffixe utilisateur** (ex. `dc=example,dc=org`) : données métier (personnes, groupes). La déclaration du suffixe dans `cn=config` ne crée pas automatiquement l'entrée racine ni l'arborescence : il faut un **LDIF** et `ldapadd` (ou équivalent) pour *peupler* l'annuaire.

Les sections suivantes enchaînent : lancement Docker, vérifications réseau, puis commandes LDAP et tests automatisés.

Chapitre 2

Installation et déploiement automatisé

2.1 Recherche de la cible à déployer

Les contraintes imposent une image basée sur Debian 12 (`debian:12-slim`), l'installation de `slapd` et `ldap-utils`, l'exposition des ports LDAP (389, 636 si TLS), et une initialisation **non interactive** par scripts Bash.

Dans le projet, la « cible » est décrite dans `projet/docker/Dockerfile` et `projet/docker-compose.yml`.

2.2 Construction et démarrage

On construit l'image et on démarre les services définis dans le compose (au minimum le service `ldap`) :

```
# Racine du depot TP-LDAP, puis repertoire compose
cd projet
docker compose up -d --build
docker ps
docker logs -f ldap
```

Équivalent sans changer de répertoire (toujours depuis la racine du dépôt) :

```
docker compose -f projet/docker-compose.yml up -d --build
```

2.3 Comportement au démarrage du conteneur

Le script `projet/docker/entrypoint.sh` :

1. garantit une configuration `cn=config` si les volumes sont vides ;
2. démarre `slapd` en arrière-plan (`ldap:///` et `ldapi:///`) sous l'utilisateur système `openldap` ;
3. attend la disponibilité du socket `ldapi` ;
4. exécute séquentiellement les scripts montés dans `/container/init.d/` (par ex. `10-init_ldap.sh`, `20-init_ldap_linux_integration.sh`, etc.).

Après les scripts, le processus principal du conteneur reste actif (boucle `sleep infinity`), ce qui maintient le conteneur en vie comme demandé par l'énoncé.

2.4 Persistance

`docker-compose.yml` déclare des volumes nommés pour `/var/lib/ldap` (données MDB) et `/etc/ldap/slapd.d` (configuration). Un redémarrage du conteneur conserve l'annuaire.

2.5 Vérifications (suffixe et service)

```
# Conteneur actif
docker ps | grep ldap

# La racine du suffixe rpond
ldapsearch -x -H ldap://localhost:389 \
  -b dc=example,dc=org -s base dn

# Authentification administrateur annuaire (RootDN)
ldapwhoami -x -D cn=admin,dc=example,dc=org -w admin
```

Résultat attendu : `ldapsearch` retourne une entrée avec `dn: dc=example,dc=org` ; `ldapwhoami` affiche `dn:cn=admin,dc=example,dc=org`.

2.6 Recherche d'informations sur la configuration (cn=config)

On inspecte la configuration dynamique via la socket locale `ldapi` et SASL `EXTERNAL` (sans mot de passe RootDN) *depuis le conteneur* :

```
docker exec ldap ldapsearch -Y EXTERNAL -H ldapi:/// \
  -b cn=config -s base dn

# Apercu de la base MDB et des olcAccess (extrait volumineux)
docker exec ldap ldapsearch -Y EXTERNAL -H ldapi:/// \
  -b cn=config -s sub "(olcDatabase=mdb)" dn olcSuffix olcRootDN
```

Interprétation : le suffixe des données utilisateur (`olcSuffix`) et le `olcRootDN` apparaissent sous l'entrée `olcDatabase={1}mdb,cn=config`. Les règles `olcAccess` y sont également listées.

2.7 Tests automatisés - `test_01_installation_deploiement.sh`

Le script vérifie que le conteneur `ldap` est bien en cours d'exécution, que `slapd` répond sur le port `389` (recherche LDAP anonyme sur le base DN), et que l'authentification `ldapwhoami` avec SASL `EXTERNAL` sur `ldapi:///` fonctionne à l'intérieur du conteneur. Il résume ensuite le nombre de tests réussis ou échoués.

```
# Depuis la racine du depot
./projet/test/test_01_installation_deploiement.sh
```

Attendu (INSTRUCTIONS)	Réalisation (dépôt)
Dockerfile Debian, slapd, ports	`projet/docker/Dockerfile`
Init non interactive, LDIF	`projet/scripts/init_ldap.sh`, etc.
`docker-compose.yml`, volumes	`projet/docker-compose.yml`
Entrypoint + maintien du conteneur	`projet/docker/entrypoint.sh`

Chapitre 3

Conception de la structure DIT

3.1 Contexte

Après reconfiguration de la base MDB et définition du suffixe dans `cn=config`, le **contenu** du DIT (racine `dc=...`, OU, utilisateurs) est créé par des fichiers **LDIF** et `ldapadd` / `ldapmodify`. La configuration du suffixe seule ne crée ni l'entrée racine du contexte ni les sous-arbres : le peuplement reste une étape distincte.

3.2 Paramètres du base DN

Les variables d'environnement du compose pilotent le DIT :

— `LDAP_BASE_DN` : suffixe (ex. `dc=example,dc=org`) ;

— `LDAP_ORGANISATION` : attribut `o` ;

— `LDAP_DOMAIN` : domaine logique (mail, dérivation du premier `dc`).

3.3 Arborescence retenue

```
dc=example,dc=org
+-- ou=people,dc=example,dc=org
|   +-- uid=thomas,ou=people,dc=example,dc=org
|   +-- uid=john,ou=people,dc=example,dc=org
+-- ou=groups,dc=example,dc=org
    +-- cn=admin_ldap,ou=groups,dc=example,dc=org
    +-- cn=developers,ou=groups,dc=example,dc=org
    +-- cn=admin_keycloak,ou=groups,dc=example,dc=org
```

NOTA : Les groupes `groupOfNames` exigent au moins un attribut `member`. Le script d'init utilise souvent un membre factice puis le remplace par les vrais utilisateurs.

3.4 Exemple de LDIF conceptuel (racine et OU)

```
dn: dc=example,dc=org
objectClass: top
objectClass: dcObject
objectClass: organization
o: Example Org
dc: example
```

```
dn: ou=people,dc=example,dc=org
objectClass: top
objectClass: organizationalUnit
ou: people
```

3.5 Vérifications

```
ldapsearch -x -H ldap://localhost:389 \
  -D "cn=admin,dc=example,dc=org" -w admin \
  -b "dc=example,dc=org" -s base dn

ldapsearch -x -H ldap://localhost:389 \
  -D "cn=admin,dc=example,dc=org" -w admin \
  -b "ou=people,dc=example,dc=org" -s one dn
```

3.6 Justification (éléments de rapport)

— **Séparation** `people` / `groups` : usage courant, compatible NSS/PAM et clients LDAP.

— **Suffixe** : aligné sur un domaine organisationnel ; évolutif (sous-OU par site, `ou=services`, etc.).

3.7 Tests automatisés - `test_02_conception_dit.sh`

Le script contrôle, par des `ldapsearch` en lecture simple sur `localhost:389`, que le base DN existe, que les unités organisationnelles `ou=people` et `ou=groups` sont présentes, que les entrées `uid=thomas` et `uid=john` se trouvent sous `ou=people`, et que les groupes `admin_ldap`, `developers` et `admin_keycloak` existent sous `ou=groups`.

```
./projet/test/test_02_conception_dit.sh
```

Chapitre 4

Discrétisation des rôles et ACL

4.1 Groupes et rôles

Groupe	Rôle
`admin_ldap`	Administration déléguée de l'annuaire (ACL `manage`)
`developers`	Groupe fonctionnel exemple
`admin_keycloak`	Rôle procédural (qui administre Keycloak côté org.)

`thomas` est membre de `admin_ldap` ; `john` de `developers`.

4.2 Compte `cn=admin` et administration courante

Le **RootDN** `cn=admin,dc=example,dc=org` reste le compte racine de la base MDB : OpenLDAP l'autorise toujours pour les opérations d'administration au niveau serveur. L'énoncé demande de **ne pas** l'utiliser pour l'administration *courante* : les ACL confèrent les droits d'administration aux membres de `admin_ldap`. En pratique, les scripts d'init peuvent encore employer `cn=admin` pour des opérations techniques ; la délégation `admin_ldap` est néanmoins en place.

4.3 Où sont les ACL ?

Les règles `olcAccess` sont portées par `olcDatabase={1}mdb,cn=config`. Leur application depuis le conteneur utilise typiquement :

```
ldapmodify -Y EXTERNAL -H ldapi:/// -f fichier.ldif
```

4.4 Règles déployées dans le dépôt

Les ACL sont posées par le script `projet/scripts/init_ldap.sh` (après le peuplement du DIT sur le fournisseur) : d'abord une règle minimale pour `cn=admin`, puis un bloc *add* de trois directives pour la délégation `admin_ldap`. Pour le suffixe par défaut `dc=example,dc=org`, cela correspond conceptuellement au LDIF suivant (deux modifications successives sur la même entrée `cn=config`) :

```
dn: olcDatabase=\{1\}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: to * by dn.exact=cn=admin,dc=example,dc=org manage by * read

dn: olcDatabase=\{1\}mdb,cn=config
```

```
changetype: modify
add: olcAccess
olcAccess: to attrs=userPassword by
    group.exact=cn=admin_ldap,ou=groups,dc=example,dc=org write by self write by
    anonymous auth by * none
olcAccess: to dn.base="" by * read
olcAccess: to * by group.exact=cn=admin_ldap,ou=groups,dc=example,dc=org manage by
    self write by users read by * none
```

Lecture : le RootDN conserve le `manage` global ; les mots de passe sont gérés avec un cas particulier (groupe `admin_ldap`, soi-même, auth anonyme) ; le reste du DIT est en `manage` pour `admin_ldap`, `write` pour l'entrée de l'utilisateur connecté, `read` pour les utilisateurs authentifiés (`users`), et refus pour les autres.

4.5 Validation manuelle recommandée

1. Avec `john`, tenter un `ldapadd` d'un utilisateur test sous `ou=people` : refus probable (non administrateur).
2. Avec `thomas` (membre de `admin_ldap`), réessayer : succès attendu selon les règles finales.

Exemple (fichier `entree-test.ldif` à adapter) :

```
ldapadd -x -H ldap://localhost:389 \
  -D "uid=thomas,ou=people,dc=example,dc=org" -w thomas123 \
  -f entree-test.ldif
```

NOTA : L'**ordre** des directives `olcAccess` est déterminant. Avant modification manuelle : lister les règles (`ldapsearch -Y EXTERNAL -H ldapi:/// -b cn=config` dans le conteneur).

4.6 Tests automatisés - `test_03_roles_acl.sh`

Le script exécute des `ldapwhoami` pour le RootDN, `thomas` et `john` ; vérifie l'appartenance de `thomas` à `admin_ldap` et `admin_keycloak`, et celle de `john` à `developers` (attributs `member` ou `memberUid` selon le schéma effectif) ; contrôle enfin que `thomas` et `john` peuvent chacun lire l'entrée racine du suffixe lorsqu'ils sont authentifiés.

```
./projet/test/test_03_roles_acl.sh
```

Chapitre 5

Intégration Linux (NSS / PAM)

5.1 Finalité

— **NSS** : résoudre utilisateurs et groupes via LDAP ;

— **PAM** : authentifier contre LDAP ;

— **Contrôles** : `getent passwd`, `getent group`, et si possible `su` / SSH.

5.2 Prérequis annuaire : attributs POSIX

Les utilisateurs créés comme `inetOrgPerson` doivent être enrichis en `posixAccount` (`uidNumber`, `gidNumber`, etc.) et les groupes en `posixGroup` (`gidNumber`, `memberUid`). Le script `projet/scripts/init_ldap_l` (exécuté après `init_ldap.sh`) réalise cette étape dans le conteneur `ldap`.

5.3 Configuration dans le conteneur

Tout est écrit par `projet/scripts/init_ldap_linux_integration.sh` (service `ldap`, rôle fournisseur uniquement) :

— `/etc/nsswitch.conf` : `passwd`, `group` et `shadow` passent par `files` puis `ldap` ; `netgroup` en `ldap`.

— `/etc/ldap/ldap.conf` : `BASE` = suffixe, `URI ldap://localhost:389`.

— PAM : `/etc/pam.d/common-auth`, `common-account`, `common-password`, `common-session` - `pam_ldap.so` en `sufficient` avec repli `pam_unix.so` ; session avec `pam_mkhomedir.so`.

— **nslcd** : `/etc/nslcd.conf` (`uri`, `base`, filtres `posixAccount` / `posixGroup`, `binddn`/`bindpw` vers `cn=admin`, ...).

— **libpam-ldap** : le script complète aussi `/etc/ldap.conf` (fichier utilisé par certains modules PAM) en cohérence avec l'URI et le suffixe.

Le conteneur s'appuie sur **nslcd** + **libnss-ldap** plutôt que SSSD, pour rester simple dans l'image Debian du TP.

5.4 Vérifications dans le conteneur

```
docker exec ldap getent passwd thomas
docker exec ldap getent passwd john
docker exec ldap getent group admin_ldap
docker exec ldap getent group developers
```

5.5 Authentification (`su`, SSH)

Dans une image minimale, `su - thomas` ou SSH peuvent nécessiter une configuration supplémentaire (serveur SSH, répertoires home). L'important pour le TP est de montrer NSS + PAM cohérents ; un test manuel `docker exec -it ldap su - thomas` peut être documenté selon l'environnement.

5.6 Tests automatisés - `test_04_integration_linux.sh`

Le script lance `getent passwd` et `getent group` *dans* le conteneur `ldap` : il vérifie que `thomas` et `john` sont résolus comme comptes POSIX, et que les groupes `admin_ldap` et `developers` apparaissent côté NSS ; certaines étapes affichent la ligne complète retournée par `getent` pour inspection.

```
./projet/test/test_04_integration_linux.sh
```

Chapitre 6

Intégration Keycloak (OpenID)

6.1 Déploiement Keycloak

Le service `keycloak` est défini dans `projet/docker-compose.yml` (image `quay.io/keycloak/keycloak:26.2`
Ports hôte : **8090** → HTTP Keycloak, **9001** → management.

```
cd projet
docker compose up -d
```

Attendre 30 à 60 secondes au premier lancement. Console d'administration : http://localhost:8090/admin/ - compte bootstrap par défaut : `admin` / `admin`.

6.2 User Federation LDAP

Depuis le conteneur Keycloak, l'URI vers l'annuaire est `ldap://ldap:389` (nom du service Docker). Le script idempotent `projet/scripts/configure_keycloak_ldap.sh` :

— attend la disponibilité de Keycloak (`KEYCLOAK_URL`, défaut `http://localhost:8090`) ;

— obtient un jeton `admin-cli` sur le realm `master` ;

— crée le realm `tp-ldap` ;

— crée le composant LDAP et lance une synchronisation complète.

```
# Depuis la racine du depot
bash projet/scripts/configure_keycloak_ldap.sh
```

Variables utiles en surcharge : `KEYCLOAK_URL`, `KEYCLOAK_ADMIN`, `KEYCLOAK_ADMIN_PASSWORD`, `KEYCLOAK_REALM`, `LDAP_BIND_DN`, `LDAP_BIND_PW`, `LDAP_USERS_DN`.

6.3 Vérifications

— `docker compose ps` : services `ldap` et `keycloak` actifs.

— Dans le realm `tp-ldap`, onglet *Users* : présence de `thomas` et `john` après synchronisation.

— Connexion avec identifiants LDAP : `thomas` / `thomas123`.

— Rôles applicatifs : créés et assignés **dans Keycloak**, pas dupliqués dans LDAP (conformément à l'énoncé).

6.4 Tests automatisés - `test_05_keycloak.sh`

Le script s'assure que le conteneur `keycloak` tourne et que l'URL Keycloak répond ; il invoque ensuite `configure_keycloak_ldap.sh` (realm `tp-ldap`, fédération LDAP, synchronisation). Enfin, via l'API d'administration Keycloak (jeton `admin-cli`, requêtes `curl` et filtrage `python3`), il vérifie qu'un utilisateur de nom exact `thomas` et un `john` sont bien présents dans le realm après import depuis LDAP.

```
./projet/test/test_05_keycloak.sh
```

Chapitre 7

Réplication LDAP (RW / RO)

7.1 Architecture

Rôle	Service	Port hôte	Comportement
Fournisseur RW	`ldap`	389	syncprov, `olcServerID: 1`
Réplica RO	`ldap-replica`	1389 → 389 interne	`olcReadOnly: TRUE`, syncrepl

Le réplica se connecte au fournisseur à `ldap://ldap:389` sur le réseau Compose. Variable `LDAP_SERVICE_ROLE=consumer` sur le réplica.

7.2 Séquence d'initialisation (résumé)

— **Fournisseur** : DIT + ACL ; chargement module `syncprov`, overlay sur la MDB, index `entryUUID`, `olcServerID: 1`.

— **Réplica** : base MDB et ACL alignées ; pas d'import local du jeu de données ; `olcSyncrepl` vers le fournisseur ; `olcReadOnly: TRUE` pour refuser les écritures *clients*.

Les healthchecks attendent un fichier `/container/.ldap-init-complete` et des contrôles LDAP pour éviter de tester un annuaire encore en construction.

7.3 Vérifications manuelles

```
# Donnees visibles sur le replica apres synchro
ldapsearch -x -H ldap://localhost:1389 \
  -D cn=admin,dc=example,dc=org -w admin \
  -b dc=example,dc=org "(uid=thomas)" dn mail

# Ecriture sur le fournisseur (OK)
ldapmodify -x -H ldap://localhost:389 \
  -D cn=admin,dc=example,dc=org -w admin <<'EOF'
dn: uid=thomas,ou=people,dc=example,dc=org
changetype: modify
replace: description
description: test replication
EOF

# Relecture sur le replica (la valeur doit suivre apres synchro)
ldapsearch -x -H ldap://localhost:1389 \
  -D cn=admin,dc=example,dc=org -w admin \
  -b uid=thomas,ou=people,dc=example,dc=org -s base description
```

```
# Tentative d'ecriture sur le replica : echec attendu
```

7.4 Tests automatisés - `test_06_replication.sh`

Le script vérifie que `ldap-replica` est démarré, que le fournisseur répond sur `389` et le réplica sur `1389`, puis attend (boucle temporisée) que l'entrée `uid=thomas` apparaisse sur le réplica. Il modifie l'attribut `description` de `thomas` sur le **fournisseur**, contrôle que la même valeur est lue sur le **réplica**, et vérifie qu'un `ldapmodify` sur le réplica échoue bien (base en lecture seule pour les clients). Il supprime enfin `description` sur le fournisseur pour nettoyer.

```
./projet/test/test_06_replication.sh
```

7.5 Dépannage (rappel)

Si le fournisseur reste `unhealthy`, vérifier les logs `docker logs ldap` et la présence d'une entrée `olcSyncProvConfig` sous `cn=config`. En cas de doute sur l'état des données : `docker compose down -v` puis remontée complète.

Chapitre 8

Fédération LDAP (méta-annuaire)

8.1 Principe

Plusieurs annuaires distincts (simulation d'entités) sont agrégés derrière un **point d'accès unique** : le méta-annuaire OpenLDAP (**back-meta**), avec remapping de suffixe (**suffixmassage**).

8.2 Services dans le dépôt

Service	Suffixe réel	Port hôte
`ldap`	`dc=example,dc=org`	389
`ldap-acme`	`dc=acme,dc=com`	2389
`ldap-meta`	virtuel `o=federation`	3389

Sous `o=federation`, les sous-arbres virtuels `ou=example-org,o=federation` et `ou=acme-corp,o=federation` pointent respectivement vers `dc=example,dc=org` et `dc=acme,dc=com`. Les deux annuaires contiennent des utilisateurs `thomas` / `john` avec des adresses `mail` différentes (`thomas@example.org` vs `thomas@acme.com`).

8.3 Vue côté client

Bind administrateur sur le méta : `cn=admin,o=federation` / `admin`.

```
ldapsearch -x -H ldap://localhost:3389 \
  -D cn=admin,o=federation -w admin \
  -b o=federation -s sub "(uid=thomas)" dn mail
```

Résultat attendu : deux entrées (chemins distincts sous `ou=example-org` et `ou=acme-corp`), distinguables par `mail`.

8.4 Tests automatisés - `test_07_meta_annuaire.sh`

Le script contrôle que les conteneurs `ldap-meta` et `ldap-acme` sont actifs, que le méta répond sur le port `3389` et annonce le contexte de nommage `o=federation`. Il vérifie que l'on retrouve `thomas@example.org` sous l'arborescence virtuelle liée à l'annuaire `example` et `thomas@acme.com` sous celle liée à `acme`. Enfin, une recherche sous `o=federation` pour `(uid=thomas)` doit retourner **deux** entrées distinctes (une par source).

```
./projet/test/test_07_meta_annuaire.sh
```

NOTA : Le paquet `slapd` dans l'image Debian peut afficher une version 2.5.x ; les mécanismes `meta` et `suffixmassage` restent alignés sur la famille OpenLDAP visée par le TP.

Chapitre 9

Bilan : suite de tests complète

Le script `test_all_implemented.sh` enchaîne les sept scripts décrits dans les chapitres précédents, dans l'ordre : installation, DIT, rôles/ACL, intégration Linux, Keycloak, réplication, méta-annuaire. À la fin, il imprime une ligne de synthèse par script (*OK* ou *échec*). Il suppose que la stack Docker correspondante est déjà démarrée (`ldap`, `ldap-replica`, `keycloak`, `ldap-acme`, `ldap-meta` selon les parties que vous validez).

```
# Racine du depot TP-LDAP
./projet/test/test_all_implemented.sh
```

Avant de lancer la suite complète : `docker compose up -d` (ou équivalent) depuis `projet/`, puis attendre le passage des conteneurs en état sain lorsque des healthchecks sont définis.

Annexe A - Table des ports

Port hôte	**Usage**
389	LDAP fournisseur (`ldap`)
1389	LDAP réplica (`ldap-replica`)
2389	Second annuaire (`ldap-acme`)
3389	Méta-annuaire (`ldap-meta`)
8090	Keycloak HTTP
9001	Keycloak management

Annexe B - Réinitialisation et nettoyage

```
cd projet
# Arret + suppression des volumes (donnees perdues)
docker compose down -v
docker compose up -d --build
```

Nettoyage plus agressif (images locales, etc.) : `docker compose down -v --rmi local`.

Si un conteneur orphelin bloque le nom `ldap` :

```
docker rm -f ldap 2>/dev/null || true
cd projet && docker compose down -v --remove-orphans
docker compose up -d --build
```

Annexe C - Références

— `INSTRUCTIONS.md` - objectifs officiels du TP.
— `README.md` - démarrage rapide, structure du dépôt.
— `projet/test/README.md` - détail des scripts de test.
— Documentation OpenLDAP 2.6 : https://www.openldap.org/doc/admin26/guide.html